CSC-239
Java Programming:
Chapter 8: Multidemsional Arrays

October 1, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Reminder: Quiz 2

- Quiz 2 will begin on Monday, October 6.
  - Online (Moodle) Quiz.
  - The Quiz will open on Monday at 9:00 p.m. and remain open for approximately one week.
  - Each student will have two hours to complete the Quiz, from the time that they begin.
- The material covered on Quiz 2 will be
  - The lectures of September 10 through 29.
  - Chapters 4 through 7.
  - The quiz will include True / False, Multiple-Choice questions.

Two More Reminders:

1. **Project 1** is due on Wednesday, October 15.
2. **Remember** these details (in the ***Syllabus***) about Lab Exercise due dates:
   - Students should submit the source code for their **Lab** solution to *Moodle* as soon as **they** feel it is working correctly.
   - Late penalties are imposed based on the **submit** date, not the date of demonstration during class.
   - The instructor maintains a record of whether each student has ***successfully*** demonstrated the solution to each Lab Exercise. This data becomes a significant part of the student's final course grade.

Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

- *Grader Than* Support E-mail:

  support@graderthan.com

Multidimensional Arrays

- Implemented as an "array of arrays":

```
// Declare array reference variable
```

dataType`[][]` *refVar*`;`

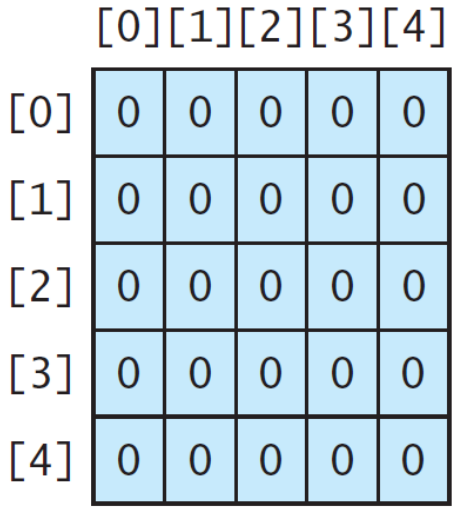
```
// Create array, set the reference variable
```

refVar `= new` *dataType*`[`*rows*`][`*columns*`];`

- For example:

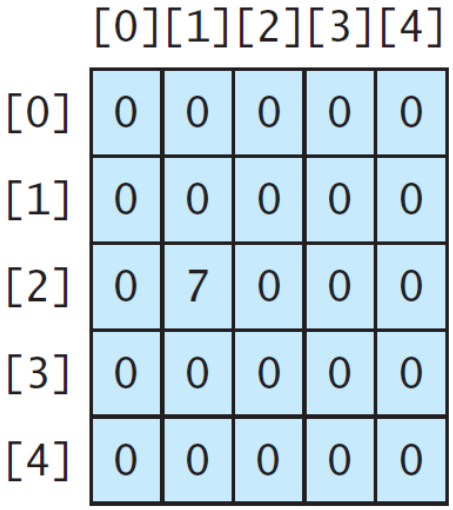
```
double[][] matrix;
matrix = new double[5][10];
```

Two-dimensional Array Illustration



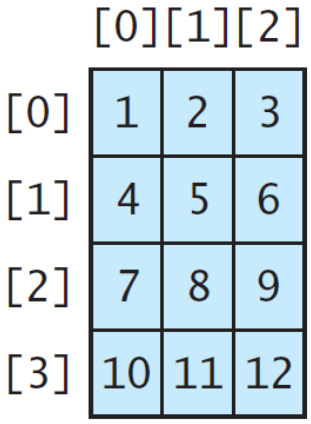

```
matrix = new int[5][5];
```

(a)




```
matrix[2][1] = 7;
```

(b)




```
int[][] array = {
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

(c)

matrix.length? 5

matrix[0].length? 5

array.length? 4

array[0].length? 3

Declaring, Creating, and Initializing Using Shorthand Notations

You can also use an *array initializer* to declare, create and initialize a two-dimensional array.

For example:

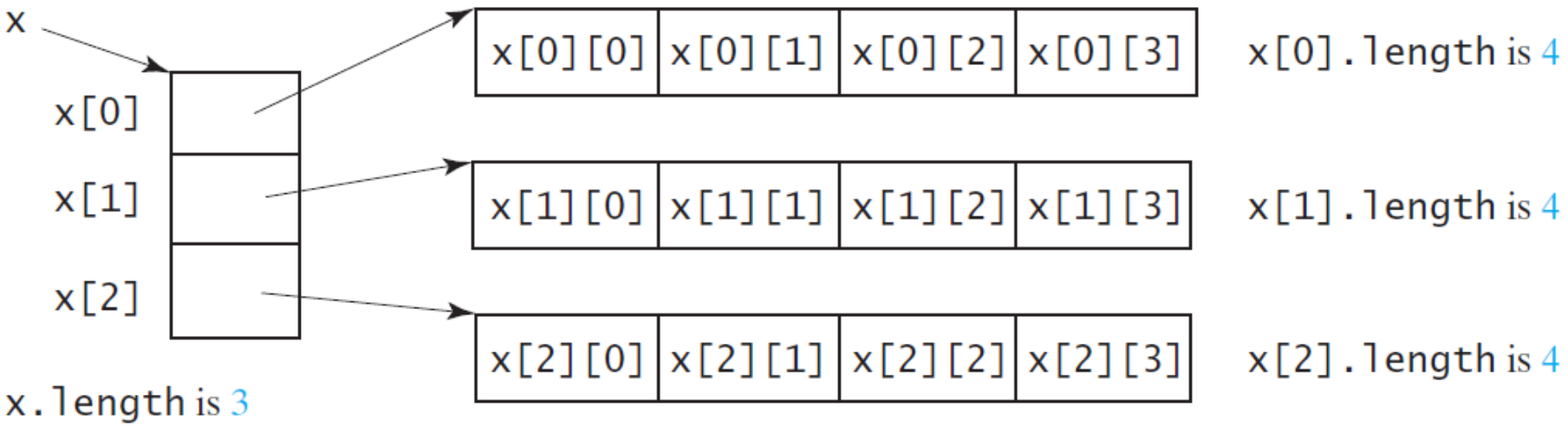
```
int[][] array = {
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

Same as

```
int[][] array = new int[4][3];
array[0][0] = 1; array[0][1] = 2; array[0][2] = 3;
array[1][0] = 4; array[1][1] = 5; array[1][2] = 6;
array[2][0] = 7; array[2][1] = 8; array[2][2] = 9;
array[3][0] = 10; array[3][1] = 11; array[3][2] = 12;
```

Lengths of Two-dimensional Arrays

```
int[][] x = new int[3][4];
```



Lengths of Two-dimensional Arrays, cont.

```
int[][] array = {
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

array.length:	4
array[0].length:	3
array[1].length:	3
array[2].length:	3
array[3].length:	3

If we attempt to access the array beyond its bounds, then a *runtime exception* occurs:

ArrayIndexOutOfBoundsException

Ragged Arrays

- Each row in a two-dimensional array is itself an array.
- So, each *row* can have a different length.
- Such an array is known as *a ragged array*.

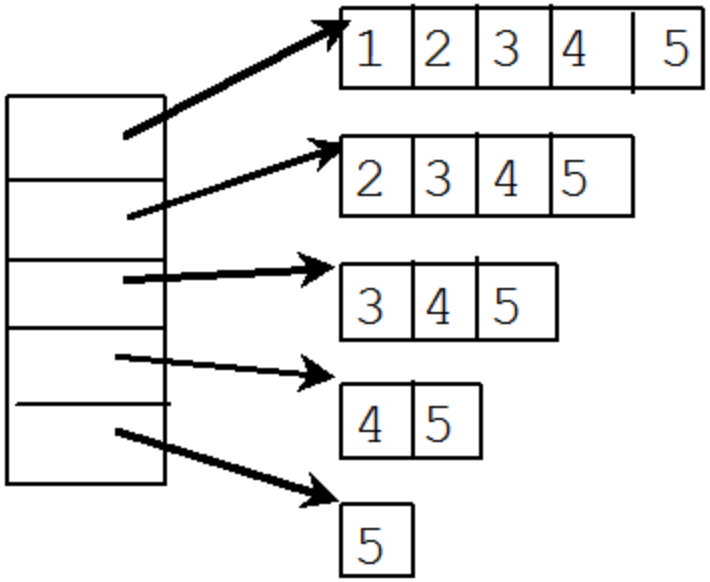
Ragged Array Example (1)

```
int[][] triangleArray = {
   {1, 2, 3, 4, 5},
   {2, 3, 4, 5},
   {3, 4, 5},
   {4, 5},
   {5}
};
```

```
triangleArray.length is 5
triangleArray[0].length is 5
triangleArray[1].length is 4
triangleArray[2].length is 3
triangleArray[3].length is 2
triangleArray[4].length is 1
```

Ragged Array Example (2)

```
int[][] triangleArray = {
   {1, 2, 3, 4, 5},
   {2, 3, 4, 5},
   {3, 4, 5},
   {4, 5},
   {5}
};
```



Example: Input to a 3x5 array

```
java.util.Scanner input =
                    new Scanner(System.in);
int[][] matrix = new int[3][5];

System.out.println("Enter " + matrix.length
    + " rows and " + matrix[0].length + "
    columns: ");

for (int row=0; row<matrix.length; row++) {
   for (int column = 0;
           column < matrix[row].length;
                column++) {
      matrix[row][column] = input.nextInt();
   }
}
```

Example: Print a 2-dimensional Array

```
for (int row=0; row < matrix.length; row++){
    for (int column = 0;
         column < matrix[row].length;
         column++) {
     System.out.print(
           matrix[row][column]+ " ");
    }

    System.out.println();
}
```

Pass 2-dimensional Array to a Method (1)

```
import java.util.Scanner;

public class PassTwoDimensionalArray {
  public static void main(String[] args) {
    Scanner input = new Scanner(System.in);

    int[][] m = new int[3][4];
    System.out.println("Enter " + m.length +
    " rows, and " +m[0].length + " columns: ");
    for (int i = 0; i < m.length; i++) {
      for (int j = 0; j < m[i].length; j++) {
        m[i][j] = input.nextInt();
      }
    }

    System.out.println("\nSum of all elements is
    " + sum(m));
  } // end main
```

Pass a 2-dimensional Array to a Method (2)

```
  public static int sum(int[][] m) {
    int total = 0;
    for (int row = 0; row < m.length; row++)
    {
      for (int column = 0; column <
    m[row].length; column++) {
        total += m[row][column];
      }
    }

    return total;
  } // end sum
} // end class PassTwoDimensionalArray
```

N-dimensional Arrays

- In Java, you can create n-dimensional arrays for any integer n.

- The way to declare two-dimensional array variables and create two-dimensional arrays can be generalized to declare n-dimensional array variables and create n-dimensional arrays for n >= 3.

Example: 3-Dimensional Array

```
//      Dimensions of array
final int ROWS = 3;
final int COLS = 5;
final int LAYERS = 2;

double[][][] data =
    new double[ROWS][COLS][LAYERS];
```

PrimeNumbers Lab Exercise

- Lab Exercise 8a involves calculating the first 50 prime numbers.
- A "prime" number is a positive integer which is evenly divisible *only* by itself and 1:

  That is, for any positive integers **n** and **d**, if

  n, d ≠ 1 and **d ≠ n**

  and the integer division **n / d** has remainder = 0,

  then **n** is *not* a prime number.

How can we generate Prime Numbers?

- **Sample program**: Section 5.14 of the textbook (pages 191-193) develops a simple program to print the first 50 prime numbers, over 5 lines of output (10 primes per line).
- **Design**: The problem can be broken into the following tasks:
  - For number = 2, 3, 4, 5, 6, ..., test whether the number is prime.
  - Count the prime numbers, as they are found.
  - Print each prime number on the screen, formatted as 5 lines of text with 10 numbers per line.

Prime Number Algorithm

- Set the number of prime numbers desired to a constant called NUMBER_OF_PRIMES.
- Use a **`count`** variable to track the number of primes found so far. Initially, **`count = 0`**.
- Use a **`number`** variable to represent each "candidate" value we are testing. Initially, **`number = 2`**.
- Pseudocode:

```
while (count < NUMBER_OF_PRIMES) {
        If ( number is a prime) {
                Display the number.
                Increment count by 1.
        }
        Increment number by 1.
}
```

Testing if a Number is Prime

- Use a **boolean** variable **`isPrime`** to indicate whether **`number`** is prime. Initially, set **`isPrime`** to true.
- Use an integer variable **`divisor`** to test if **`number`** is prime.
- The values of **`divisor`** range from **`2`** through **`number / 2`**.
- If we find one value of **`divisor`**, for which

  `number` / **`divisor`** has remainder = `0`,

  then we must set **`isPrime`** to false.
- The pseudo-code looks like:

```
for(divisor=2; divisor <= number/2; divisor++){
    If ( number % divisor == 0) {
                Set isPrime = false.
                Exit the loop.
     }
}
```

The Lab08a Exercise (1)

- Enhance an existing program called "**PrimesComparison.java**".
- It contains two methods for finding prime numbers:
  - **primesSequentialDivisors**: implements the algorithm from section 5.14 of the textbook, but also maintains a count of *how many* potential divisors were checked for each prime number eventually found.
  - **primesPrimeDivisors**: is intended to implement a more efficient algorithm, and also track how many potential divisors are required. In the "starter code", however, this method is just a stub.
  - **outputResults**: displays two 2-dimensional arrays "side-by-side" (4 columns total).

The Lab08a Exercise (2)

- The assignment of Lab08a is to implement the **primesPrimeDivisors** method:
  - The resulting list of prime numbers will be the same for both methods.
  - The **primesPrimeDivisors** method will need to test fewer **divisor** values to determine whether or not a given **number** value is prime.
  - Instead of dividing **number** by all integers from **2** to **number / 2**, divide by all known *prime numbers* less than the *square root* of the **number** value.